

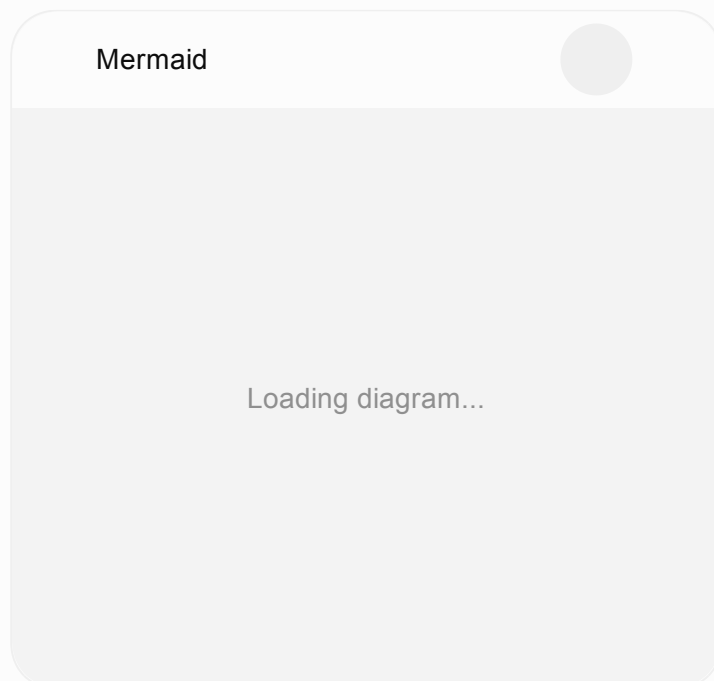
I want to develop an event registration program. The program includes user registration from a Google form link, which includes food selection. When the registration completes, the user receives a QR code and receipt from the cell phone or email. The organizer then use accumulate the information to

Show more

Worked for 40s

Yes—I can help design, code, and test the complete system. The best first version is a Google Workspace MVP that runs in a phone browser and requires no app installation.

Recommended first architecture



Google Forms should collect information, but the operational logic should reside in Google Apps Script and the response spreadsheet. Google Forms alone is not well suited to QR validation, duplicate-scan protection, or meal redemption.

Registration record

Each registration should contain separate fields such as:

Field	Purpose
Registration ID	Human-readable reference number
QR token	Random, unguessable identifier encoded in the QR
Name, email, phone	Attendee information
Event ID	Supports multiple church activities
Meal selection	Requested meal or “no meal”
Registration status	Submitted, confirmed, canceled, waitlisted
Meal ordered	Organizer confirms the meal was included in the final order
Event check-in time	Set by the check-in scanner
Meal redeemed time	Set by the lunch scanner
Operator/device	Audit trail for each scan

I recommend timestamps rather than simple scanned flags. A nonempty `meal_redeemed_time` means the meal has already been collected.

The lunch logic would be:

1. Is the QR token valid?
2. Is the registration active?
3. Did the attendee request a meal?
4. Did the organizer mark that meal as ordered?
5. Has it already been redeemed?

6. If valid, record the time and display the meal type prominently.

A duplicate scan should display “**Already redeemed at 12:18 PM**” without issuing another meal.

User experience

The attendee:

1. Opens the Google Form on a phone.
2. Registers and selects food.
3. Receives a confirmation receipt and QR code by email.
4. Optionally receives an SMS link to the QR.
5. Presents the same QR at event check-in and lunch.

The organizer:

- Views registration and meal totals.
- Filters totals by entrée, dietary restriction, and attendance status.
- Freezes or marks the final meals as ordered.
- Uses ordinary phones as QR scanners.
- Has separate **Event Check-in** and **Meal Pickup** scanner modes.
- Can search by name or registration number if a phone is lost.
- Can export records for accounting or reporting.

Email is straightforward in the Google version.

SMS requires a service such as Twilio and should be optional; an emailed mobile wallet-style page may be sufficient initially.

Designing it for later expansion

I would make the application event-driven rather than hard-code it for one event:

- An `Events` table defines dates, locations, forms, and registration limits.

- A `MealOptions` table defines food choices for each event.
- A `Registrations` table stores attendees.
- A `ScanLog` table provides an audit history.
- A `Users/Roles` table controls organizers, check-in volunteers, and meal-desk volunteers.

This makes it reusable for retreats, Sunday school, conferences, volunteer activities, transportation, childcare, and ticketed events.

The first version can use Google Sheets as the database while keeping the application logic separate. Later, the same interface can connect to PostgreSQL, Firebase, or Salesforce without changing the attendee workflow. Salesforce would be valuable if the church already uses it for contact management, reporting, or follow-up; otherwise, it may be unnecessarily complex for the MVP.

What I can build

I can produce:

- The Google Form creation/setup script
- Google Sheet schema and organizer dashboard
- Apps Script backend
- QR generation and receipt emails
- Responsive phone-based scanner pages
- Check-in and meal-redemption workflows
- Duplicate-scan and invalid-order handling
- Food-count reports and CSV export
- Role-based access and audit logs
- Automated tests and test data
- Deployment and phone-testing instructions
- A later database/Salesforce version

I can test the program logic and mobile layouts here. Deployment under your Google account requires you to authorize the Apps Script

application, and final camera testing requires opening the deployed scanner on one or more physical phones. I can provide a precise test checklist and correct any issues found.

Before I begin the MVP, I need four decisions:

1. Is the “receipt” only a registration confirmation, or will the system collect payment?
2. Should version 1 send QR codes by email only, or by both email and SMS?
3. Can one registration include a family/group, or is each QR for one person?
4. What food choices and attendee information should the initial Google Form contain?

1. Let's start with a registration confirmation, not payment collection. We will consider paid conference later on. 2. QR codes sent by both email and SMS. 3. Each QR for one person. 4. Food choices: the event organizer will input the food names. They are customized per event.

Worked for 4m 56s

I built the first Google Apps Script MVP:

Download [church-event-registration-mvp.zip](#)

It includes:

- Configurable Google Form and meal choices
- One QR token per attendee

- Email confirmation with QR
- SMS receipt link through Twilio
- Phone-based QR scanner
- Separate event check-in and meal-pickup modes
- `meal_selected`, `meal_ordered`, and `meal_redeemed_at` tracking
- Duplicate-scan protection
- Food summary and scan audit log
- Setup/deployment instructions
- Automated core tests, which pass

The Google Form initially remains closed. After deployment, you configure the web-app URL, private operator PINs, event information, meals, and Twilio credentials, then activate registration.

Do not send Twilio credentials through chat; enter them directly into the Apps Script project as described in the README.

The next stage is to deploy this under your Google account and run the included iPhone/Android acceptance checklist. Once deployed, I can help troubleshoot each test and refine the organizer interface.

Please provide step by step instructions on creating a new standalone Apps script project at "script.google.com". I have a Google account and I am using Window's laptop. I have downloaded your codes. Please help me with the first milestone on create google form and database using the App. I will then ask how to access from the cell phone to

the form and register from there.

Worked for 55s

For this first milestone, we will:

1. Create a standalone Apps Script project.
2. copy the required source files into it;
3. run `setupApplication()` ;
4. verify that the Google Form and spreadsheet database were created;
5. customize the event and food choices.

Do not activate the form or submit registrations yet.

QR delivery requires the web-app deployment, which will be our next milestone.

Part 1: Extract the downloaded package

1. Locate `church-event-registration-mvp.zip` in Windows File Explorer.
2. Right-click the ZIP file.
3. Select **Extract All...**
4. Select a convenient folder and click **Extract**.
5. Open the extracted `church-event-registration` folder.

You should see these files:

- `Core.gs`
- `Setup.gs`
- `Registration.gs`
- `Scanner.gs`
- `Web.gs`
- `Scanner.html`
- `Receipt.html`
- `appsscript.json`
- `README.md`
- `package.json`

For this milestone, only the five `.gs` files and two `.html` files must be copied into Google Apps Script.

Do not copy these into Apps Script:

- `package.json`
- `.clasp.json.example`
- `tests/core.test.js`
- `README.md`

Part 2: Create the standalone Apps Script project

1. Open Chrome on your Windows laptop.
2. Visit script.google.com.
3. Sign in with the Google account that should own the registration system.
4. Click **New project** near the upper-left corner.
5. The Apps Script editor will open with a default file named `Code.gs`.
6. Click **Untitled project** at the upper-left.
7. Rename it:

`Church Event Registration App`

8. Click **Rename**.

You now have a standalone Apps Script project.

Part 3: Remove the example code

The editor initially displays something similar to:

JavaScript

```
function myFunction() {  
  
}
```

You can either leave the `Code.gs` file empty or delete it.

To delete it:

1. In the **Files** panel on the left, place the mouse over `Code.gs`.
2. Click the three-dot menu.
3. Select **Delete**.

4. Confirm the deletion.

If Apps Script does not allow the only file to be deleted, first create `Core.gs` using the next section, and then delete `Code.gs`.

Part 4: Add the five script files

For every `.gs` file, create a corresponding Apps Script file.

Add `Core.gs`

1. Click the **+** next to **Files**.
2. Select **Script**.
3. Enter:

`Core`

4. Click Enter. Apps Script automatically adds `.gs`.
5. On your laptop, open the downloaded `Core.gs` using Notepad or Visual Studio Code.
6. Press `Ctrl+A` to select all the text.
7. Press `Ctrl+C` to copy it.
8. Return to the Apps Script browser window.
9. Delete any existing text in `Core.gs`.
10. Press `Ctrl+V` to paste the code.

Repeat the same procedure for:

- `Setup.gs`
- `Registration.gs`
- `Scanner.gs`
- `Web.gs`

Your Apps Script file list should now contain:

```
Core.gs
Setup.gs
Registration.gs
Scanner.gs
Web.gs
```

The order displayed in the editor does not matter.

Part 5: Add the two HTML files

HTML files must be created using **HTML**, not **Script**.

Add `Scanner.html`

1. Click the **+** next to **Files**.
2. Select **HTML**.
3. Enter:

`Scanner`

4. Press Enter.
5. Open the downloaded `Scanner.html` on your laptop.
6. Copy its complete contents.
7. Return to Apps Script.
8. Replace all automatically generated HTML with the downloaded content.

Repeat this process for `Receipt.html`.

The full Apps Script file list should be:

```
Core.gs
Setup.gs
Registration.gs
Scanner.gs
Web.gs
Scanner.html
Receipt.html
```

Part 6: Save and check the project

1. Click the **Save project** icon near the top, or press `Ctrl+S`.
2. Wait several seconds.
3. Look for red error indicators beside any file.

Do not add `appscript.json` yet. The default manifest is sufficient for this milestone and avoids adding web-deployment settings before we need them.

If `setupApplication` does not appear in the function selector later:

1. Save the project again.
2. Refresh the browser.
3. Check that you copied all five `.gs` files.
4. Check for red syntax-error symbols.

Part 7: Run the setup function

Near the top of the Apps Script editor is a function-selection drop-down. It might initially show a function name or say **Select function**.

1. Open the function drop-down.
2. Select:

`setupApplication`

3. Click **Run**.

The first execution will request permission to access your Google account.

Part 8: Authorize the application

A dialog should say **Authorization required**.

1. Click **Review permissions**.
2. Select the Google account that will own the application.
3. Review the permissions.

The application needs permission to:

- create and edit Google Forms;
- create and edit Google Sheets;
- install the form-submission trigger;
- send confirmation emails later;
- contact the SMS service later.

Because this is your own unpublished Apps Script application, Google might display:

Google hasn't verified this app

If that appears:

1. Confirm that the displayed application name is your `Church Event Registration App`.
2. Click **Advanced**.
3. Click **Go to Church Event Registration App (unsafe)**.
4. Review the permissions.
5. Click **Allow**.

The word “unsafe” here means the private application has not gone through Google’s public verification process. Only proceed when you created the project yourself and pasted the supplied code into it.

After authorization, the original setup execution may continue automatically. If it does not, select `setupApplication` and click **Run** again.

Wait until the execution log says:

Execution completed

Do not repeatedly run the function after it succeeds. It is designed to reuse the original Form and spreadsheet, but one successful run is sufficient.

Part 9: Find the created Form and spreadsheet

Open `Google Drive` in another browser tab.

Search for:

Church Event Registration

You should see two different files with that name:

- A purple Google Forms file
- A green Google Sheets file

The Google Apps Script project remains available at `script.google.com`.

Part 10: Verify the Google Form

Open the purple Google Form in editing mode.

It should contain:

1. **Full Name**
2. **Email Address**
3. **Mobile Phone**
4. **Food Selection**
5. **Dietary Notes**
6. **SMS Consent**

The form should currently be closed to responses.

That is intentional because the confirmation and QR web page have not yet been deployed.

The food choices initially contain sample values:

- Chicken
- Vegetarian
- No meal

We will replace them with your actual event choices.

Part 11: Verify the spreadsheet database

Open the green Google Sheet.

It should contain several tabs along the bottom:

Sheet	Purpose
Settings	Event name, date, location, and configuration
Meals	Food choices defined by the organizer
Registrations	Operational attendee records
Scan Log	Check-in and meal-scan audit trail
Food Summary	Counts requested, ordered, and redeemed

Sheet	Purpose
Form Responses 1	Original submissions recorded by Google Forms

The important distinction is:

- Form Responses 1 is the unmodified record generated by Google Forms.
- Registrations is the application's working database containing QR tokens and workflow states.

Do not manually add registrations to either sheet.

Part 12: Configure the event

Open the Settings tab. It should have key and value columns.

Replace the sample values:

Key	Example value
EVENT_ID	2026-fall-fellowship
EVENT_NAME	2026 Fall Church Fellowship
EVENT_DATE	October 17, 2026, 10:00 AM
LOCATION	Church Main Hall
ORGANIZER_EMAIL	Your organizer's email
WEB_APP_URL	Leave blank for now

Use a unique EVENT_ID . A short identifier containing letters, numbers, and hyphens is best.

Part 13: Configure food choices

Open the Meals tab.

It contains:

meal_name	active
Chicken	TRUE
Vegetarian	TRUE
No meal	TRUE

Replace the sample meal names with the actual choices for this event.

For example:

meal_name	active
Chinese chicken lunch	TRUE
Vegetarian lunch	TRUE
Children's meal	TRUE
No meal	TRUE

Important rules:

- Each meal name must be unique.
- Use `TRUE` for a choice that should appear on the Form.
- Use `FALSE` to temporarily hide a choice.
- Keep `No meal` if attendees may register without food.
- Do not change the column headings `meal_name` and `active`.

Part 14: Synchronize meals with the Form

After editing the `Meals` sheet:

1. Return to the Apps Script project.
2. Save the project if necessary.
3. Open the function drop-down.
4. Select:

`syncMealChoicesToForm`

5. Click **Run**.
6. Wait for **Execution completed**.

Return to the Google Form and refresh the page.
Under **Food Selection**, you should see your customized meal choices.

Part 15: Verify the Form–database connection

In the Google Form editor:

1. Open the **Responses** tab.
2. Look for the green Google Sheets icon.
3. The Form should already be connected to the `Church Event Registration` spreadsheet.

In Apps Script:

1. Click **Triggers**—the alarm-clock icon on the left.
2. Verify that a trigger exists with:

Property	Expected value
Function	<code>handleFormSubmit</code>
Event source	From spreadsheet
Event type	On form submit

Do not create another trigger if this one already exists.

Milestone 1 completion checklist

You have completed this milestone when:

- The standalone Apps Script project contains five `.gs` and two `.html` files.
- `setUpApplication()` finishes successfully.
- Google Drive contains the Form and spreadsheet.
- The spreadsheet contains the configuration and database tabs.
- The Form contains the six registration questions.

- Your customized meals appear under **Food Selection**.
- The `handleFormSubmit` trigger exists.
- The Form is still closed to public responses.

Do not submit a test registration yet. The current submission trigger would try to create and deliver a QR receipt, but the required web-app URL has not been configured.

Google Drive
Google Drive

I have two problems so far in Google Apps Script setup. First, Scanner.html can not be created as it complains about the same name in Scanner.gs. I can best create the name Scanner.html.html. So I suggest I use a different file name for Scanner.html (e.g., change it to Scanner1.html). The second problem is I do not see `setupApplication()` function.

Worked for 1m 17s

You are correct—the first problem is a naming defect in my original package. Google Apps Script requires every project file to have a unique base name, regardless of whether it is `.gs` or `.html`.

I corrected the package here:

Download corrected MVP v0.1.1

You do not need to restart the project. Apply these corrections to your existing project.

1. Fix the Scanner filename conflict

Delete `Scanner.html.html` if you created it:

1. Find `Scanner.html.html` in the Files panel.
2. Click its three-dot menu.
3. Select **Delete**.

Create the HTML file again:

1. Click **+** beside Files.
2. Select **HTML**.
3. Enter only:

```
ScannerPage
```

4. Do not enter `.html` ; Google adds it automatically.
5. The resulting filename should be:

```
ScannerPage.html
```

6. Open the downloaded `Scanner.html` from the original package—or `ScannerPage.html` from the corrected package.
7. Copy its entire contents into the new `ScannerPage.html` .

Now open `Web.gs` and find:

JavaScript

```
const template = HtmlService.createTemplateFromF
```

Change it to:

JavaScript

```
const template = HtmlService.createTemplateFromF
```

Save with `Ctrl+S` .

Your files should include:

```
Core.gs
Setup.gs
Registration.gs
```

```
Scanner.gs
Web.gs
ScannerPage.html
Receipt.html
```

2. Check how the filenames were entered

When creating a file, enter only its base name.

Desired result	Type selected	Name entered
Core.gs	Script	Core
Setup.gs	Script	Setup
Registration.gs	Script	Registration
Scanner.gs	Script	Scanner
Web.gs	Script	Web
ScannerPage.html	HTML	ScannerPage
Receipt.html	HTML	Receipt

If you entered `Setup.gs` when creating the file, Google may have named it `Setup.gs.gs`. If so, delete it and recreate it by entering only `Setup`.

3. Verify `setupApplication` is actually in `Setup.gs`

Click `Setup.gs`, then press `Ctrl+F` and search for:

JavaScript

```
function setupApplication()
```

Near the beginning of the file, you should see:

JavaScript

```
function setupApplication() {
  const props = PropertiesService.getScriptPrope
```

If you do not see this, the contents of `Setup.gs` were not copied completely. Delete its contents and paste the complete contents of the downloaded `Setup.gs`.

Also make sure you did not paste Markdown markers such as:

```
```javascript
```

or:

```
Only the JavaScript source should be prese
```

```
4. Save and refresh the function list
```

```
After correcting the files:
```

1. Press ``Ctrl+S``.
2. Wait approximately five seconds.
3. Refresh the browser page.
4. Wait for the Apps Script project to reo
5. Open the function drop-down next to `**R`

```
The function is listed without parentheses
```

```
```text  
setupApplication
```

Look for `setupApplication`, not `setupApplication()`.

5. Check for syntax errors

If the function still does not appear:

1. Look through the files in the left panel.
2. Check for a red error symbol beside any file.
3. Open each `.gs` file and look for red underlining.
4. Make sure no JavaScript code was accidentally placed in an HTML file.

5. Make sure `ScannerPage.html` was created using **HTML**, while `Setup.gs` was created using **Script**.

A syntax error in any `.gs` file can prevent Apps Script from rebuilding the function list.

6. Use a simple diagnostic function

If necessary, create a new script file:

1. Click **+** → **Script**.
2. Enter only:

`Diagnostic`

3. Paste:

JavaScript

```
function diagnosticTest() {  
  console.log('The project can recognize function');  
}
```

4. Save and refresh the browser.

Check the function drop-down:

- If `diagnosticTest` appears but `setupApplication` does not, the problem is specifically in `Setup.gs`.
- If neither appears, the project has an unsaved file or a syntax error elsewhere.

You may delete `Diagnostic.gs` after testing.

7. Run the setup

Once `setupApplication` appears:

1. Select `setupApplication` from the drop-down.
2. Click **Run**.
3. Complete Google's authorization prompts.
4. Wait for:

Execution completed

Then open Google Drive and search for:

Church Event Registration

You should find one Google Form and one Google Sheet. Stop there and tell me whether the execution completed or provide the exact error message if it did not.

Yes, `setupApplication()` has shown. I started the "run" button. Now, I have another message: "This App might not work as expected without providing all requested permissions". How do I give permission to run the app?

Worked for 12s

That warning normally means one or more permission checkboxes were left unselected.

Since this is the Apps Script project you created in your own Google account, you can grant the requested permissions as follows:

1. On the warning screen, click **Back** or **Review permissions**.
2. Find the page titled something like:

Select what Church Event Registration App can access
3. Check **Select all**. If there is no Select All option, individually check every requested permission.
4. Click **Continue** or **Allow**.
5. Return to the Apps Script editor.

The requested permissions may include:

- Create and edit Google Forms
- Create and edit Google Sheets
- Create Apps Script triggers
- Send confirmation email
- Connect to an external service, needed later for SMS
- View your Google account email address

After returning to the editor:

1. Select `setupApplication` again if necessary.
2. Click **Run**.
3. Watch the **Execution log** at the bottom.
4. Wait for:

Execution completed

If you already continued without all permissions

If Google does not present the permission-selection screen again:

1. Visit [Google Account third-party connections](#).
2. Find **Church Event Registration App** or the name of your Apps Script project.
3. Remove its existing access.
4. Return to Apps Script.
5. Run `setupApplication` again.
6. Select your account.
7. Check all requested permissions.
8. Click **Continue**.

Only grant these permissions because you personally created this Apps Script project and pasted the reviewed source code into it. Once execution completes, search Google Drive for `Church Event Registration`; there should be one Google Form and one Google Sheet.

From the `account.google.com`, there a bunch of permission settings. Which ones should I give it an "allow"?

Worked for 17s

Do not change general permission settings throughout `account.google.com`. Permissions should be granted only through the authorization window that appears after you run `setupApplication`.

For this application, allow these requested permissions:

- **See, edit, create, and delete your Google Forms**
- **See, edit, create, and delete your Google Sheets spreadsheets**
- **Create or manage Apps Script triggers/deployments**
- **Send email on your behalf**
- **Connect to an external service**
- **See your primary Google Account email address**

Google bundles broad wording such as “create and delete” into its Forms and Sheets permissions. The supplied program does not delete your existing Forms or spreadsheets; it creates and updates the event files.

Do not approve permissions for unrelated services such as:

- Reading Gmail messages
- Deleting Gmail messages
- Google Contacts
- Google Calendar
- Google Photos

- YouTube
- Passwords or payment information

If any unrelated permission appears, stop and send me the exact permission wording.

If you are currently on the Google Account connections page, do this:

1. Find **Church Event Registration App**.
2. Remove its existing access if it was only partially authorized.
3. Return to `script.google.com`.
4. Open your project.
5. Select `setupApplication`.
6. Click **Run**.
7. Click **Review permissions**.
8. Select your Google account.
9. On **Select what Church Event Registration App can access**, check **Select all**.
10. Click **Continue**.

Then wait for `Execution completed`.

Thanks. I have the "Execution Completed" message from the log file. Now, is there any visualization of google form from the run?

Worked for 15s

Yes. The run created a real Google Form and Google Sheet, but Apps Script does not automatically open them.

View the Form

1. Open `Google Drive`.
2. Search for:

3. You should see two files:

- Purple icon: Google Form
- Green icon: Google Sheet database

4. Double-click the purple Google Form.

The Form editor should visually display:

- Full Name
- Email Address
- Mobile Phone
- Food Selection
- Dietary Notes
- SMS Consent

You can edit titles, descriptions, colors, header images, and other visual formatting there. Avoid renaming the six question titles for now because the program uses those exact titles to process responses.

The Form is currently closed to responses intentionally. Do not activate or submit it until we configure its QR receipt web address.

View the database

Open the green spreadsheet. It should contain these tabs:

- Settings
- Meals
- Registrations
- Scan Log
- Food Summary
- Form Responses 1

You may now update the event information in `Settings` and customize the choices in `Meals`.

After modifying meals, return to Apps Script and run:

```
syncMealChoicesToForm
```

Then reopen or refresh the Google Form editor to see the new food choices.

Google Drive
Google Drive

Today 5:03 PM

I am able to test the above milestone one tasks. Can we move to the next milestone?

Worked for 35s

Yes. Milestone 2 will deploy the QR-receipt web application, activate the Google Form, and complete one registration from your phone. We will test email first; SMS and scanning can follow after this works.

Milestone 2 overview

You will:

1. Configure the event and meals.
2. create operator PINs;
3. deploy the Apps Script as a web app;
4. connect the web-app URL to the spreadsheet;
5. activate the Google Form;
6. open the Form on your phone;
7. submit one test registration;
8. receive and open the QR receipt by email.

Step 1: Confirm the corrected scanner name

Your Apps Script project should contain:

```
Core.gs  
Setup.gs  
Registration.gs  
Scanner.gs  
Web.gs
```

ScannerPage.html

Receipt.html

In `Web.gs`, confirm that this line uses

`ScannerPage :`

JavaScript

```
const template = HtmlService.createTemplateFromF
```

Save the project.

Step 2: Configure the event

Open the generated Google Sheet and select the

`Settings` tab.

Enter values similar to:

key	value
EVENT_ID	2026-church-lunch-test
EVENT_NAME	Church Lunch Test Event
EVENT_DATE	September 12, 2026, 11:00 AM
LOCATION	Church Main Hall
ORGANIZER_EMAIL	Your email address
WEB_APP_URL	Leave blank temporarily

Use a test-event name for now. We can change it later.

Step 3: Configure food choices

Open the `Meals` tab.

For example:

meal_name	active
Chicken lunch	TRUE
Vegetarian lunch	TRUE

meal_name	active
Children's meal	TRUE
No meal	TRUE

Delete unused sample rows or change their `active` values to `FALSE`.

Return to Apps Script:

1. Select `syncMealChoicesToForm`.
2. Click **Run**.
3. Wait for `Execution completed`.
4. Open or refresh the Google Form.
5. Confirm that **Food Selection** shows the new choices.

Step 4: Create check-in and meal-desk PINs

The operator PIN-setting function requires parameters, which cannot be entered through the normal Run button. Create a temporary setup function.

1. In Apps Script, click **+** → **Script**.
2. Enter:

```
InitialConfig
```

3. Paste this code:

JavaScript

```
function initializeOperatorPins() {  
  setOperatorPins('246810', '135790');  
}
```

Replace the example numbers with two private PINs:

- First PIN: event check-in volunteers
- Second PIN: meal-desk volunteers

Use different PINs containing 4–10 digits.

4. Save the project.
5. Select `initializeOperatorPins` from the function drop-down.
6. Click **Run**.
7. Wait for `Execution completed`.
8. Delete `InitialConfig.gs` so the PINs are no longer stored visibly in the source code.

The application stores hashed versions of the PINs internally.

Step 5: Deploy the web application

In the upper-right corner of Apps Script:

1. Click **Deploy**.
2. Select **New deployment**.
3. Next to **Select type**, click the gear icon.
4. Select **Web app**.
5. Enter a description such as:

`Milestone 2 test deployment`

6. For **Execute as**, select:

`Me`

7. For **Who has access**, select:

`Anyone`

Depending on your Google account, it may say **Anyone, even anonymous**. Select that if available. This allows attendees to open their QR receipt without signing into Google.

8. Click **Deploy**.
9. If authorization is requested, select your account and approve the requested permissions.
10. Copy the displayed **Web app URL**.

It should resemble:

`https://script.google.com/macros/s/LONG_DE`

Use the URL ending in `/exec` . Do not use a test URL ending in `/dev` .

Keep the deployment window open until you have copied the URL.

Step 6: Store the web-app URL

Return to the Google Sheet.

In the `Settings` tab, paste the complete `/exec` URL into the value cell beside:

WEB_APP_URL

Do not put the Google Form URL in this field. This field requires the Apps Script web-app URL.

Step 7: Test the receipt web application

Open a new browser tab and paste the web-app URL.

The page should display the scanner interface with:

- Event name
- Event check-in heading
- Operator PIN field
- QR scanner area
- Manual code field

The browser might request camera permission. You may leave it denied for now because scanning is a later milestone.

Seeing this page confirms that the web application is deployed correctly.

Step 8: Activate registration

Return to Apps Script:

1. Select `activateRegistration` .
2. Click **Run**.

3. Wait for:

Execution completed

This function verifies that:

- the web-app URL is configured;
- the operator PINs exist;
- the food choices are synchronized;
- the Google Form can accept responses.

Step 9: Obtain the attendee Form link

Open the Google Form editor.

Depending on the current Google Forms interface, you may see either **Publish** or **Send**.

If you see Publish

1. Click **Publish**.
2. Set responder access to **Anyone with the link** if prompted.
3. Complete the publication.
4. Click the responder-link or copy-link control.

If you see Send

1. Click **Send**.
2. Select the link icon.
3. Optionally check **Shorten URL**.
4. Click **Copy**.

The Form link usually resembles:

`https://docs.google.com/forms/d/e/.../view`

This is the link attendees use. It is different from the Apps Script `/exec` URL.

Ensure the Form does not require attendees to belong to your Google organization unless that is intentional.

Step 10: Open the registration Form on your phone

The easiest method is:

1. Email the Google Form link to yourself.
2. Open that email on your phone.
3. Tap the link.
4. Confirm that the Form is readable without zooming.
5. Confirm that the customized food choices appear.

For this first test, use:

- Your actual name
- An email address you can check
- Your mobile number
- One meal choice
- Optional dietary notes
- The required SMS-consent checkbox

Submit the Form once.

SMS will not be sent yet unless Twilio has already been configured. That is acceptable for this milestone.

Step 11: Verify the database

Open the Google Sheet.

Check `Form Responses 1`. It should have one new row containing the original Form response.

Check `Registrations`. It should have one new row containing fields such as:

- `registration_id`
- `qr_token`
- attendee name
- email
- phone
- meal selection
- `ACTIVE` status
- `meal_ordered` set to `FALSE`

- empty check-in and redemption timestamps

This confirms that the Form submission trigger processed the registration.

Step 12: Verify the email receipt

Check the submitted email address, including its spam folder.

The confirmation should include:

- Event name
- Attendee name
- Date and location
- Food choice
- Registration number
- QR code
- Mobile QR-receipt link

Open the receipt link on your phone. It should display the QR code and registration information.

Some email applications initially block external images. If the QR image does not appear inside the email, select **Display images** or open the mobile QR-receipt link.

If the email does not arrive

In Apps Script:

1. Open **Executions** in the left panel.
2. Find the most recent `handleFormSubmit` execution.
3. Check whether its status is **Completed** or **Failed**.
4. If it failed, click the execution and copy the exact error message.

Do not submit many additional registrations until the first submission completes correctly.

Milestone 2 completion criteria

Milestone 2 is complete when:

- The `/exec` web-app URL opens.
- The Google Form accepts responses.
- The Form opens on your phone.
- One test response appears in `Form Responses 1`.
- One processed record appears in `Registrations`.
- A confirmation email arrives.
- The QR-receipt link opens on your phone.

ChatGPT can make mistakes. Workspace data isn't used to train models.